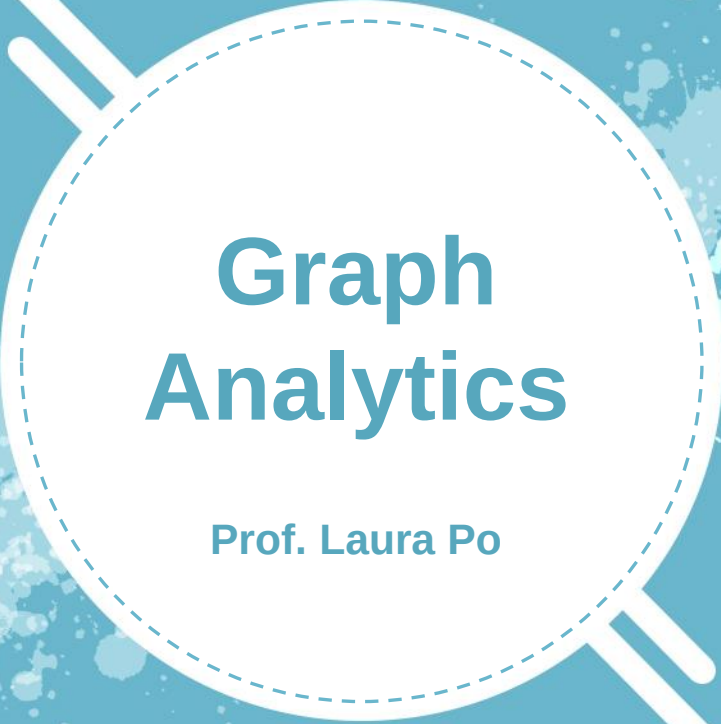
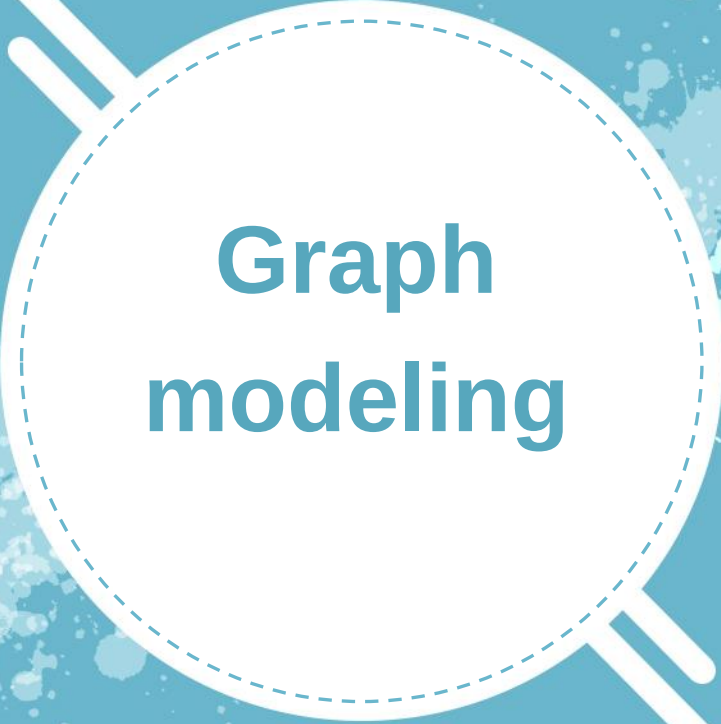




Graph Analytics

Prof. Laura Po



Graph modeling

OBJECTIVES

- Defining project goals and terminology with business or end users
- Building a **conceptual data model** for the entities and their relationships
- Translating a conceptual data model into a **graph data model**
- Comparing graph data modeling concepts to relational data modelling concepts
- Constructing the graph data model for our *social network use case*

Data Modeling

- **Data modeling** is the process of translating real-world entities and relationships into equivalent software representations. The extent to which we achieve accurate software representations of these real-world items dictates how well we address the intended problem.
- In the graph data modeling process, we must shift from an “entity first” mindset to an “entity and relationship” mindset.
- The ER model can be a good data model to use!

Data modeling steps

Four-step process for data modeling

1. **Understand the problem:** specified the common terms and the core access patterns of users.
2. **Create a conceptual model:** drawing a diagram that provide a high-level picture of the problem domain from the business perspective.
3. **Create a logical data model:** defining the vertices and the edges, as well as specifying the properties on those. 
4. **Test the model:** verify that our developed model satisfies the defined problem, that the entities and relationships needed to answer our user's questions exist, and that these are properly named (validating coherence in the three previous steps)

Most graph databases are schemaless: there is no predefined schema that dictates how nodes and relationships should be defined; nodes and relationships can be added, modified, or removed on the fly.

Advantageous in situations where the data is constantly evolving.

Neo4j is considered to be **schemaless**. It allows users to define the structure of nodes and relationships using labels and relationship types, but it does not enforce a rigid schema. Users can add, modify, or remove labels and relationship types at any time, without needing to modify the database schema.

1 - Understand the problem

Categories of questions that help gain a clear view of the problem

- Domain and scope: What will the app/the graph do for its users?
- Business entity: What are the fundamental building blocks of our application? How are these related to one another?
- Functionality: How are people going to use the system?

Example

- Domain and scope: What will the app/the graph do for its users?
 - ***DiningByFriends*** provides users with personalized restaurant recommendations. When using DiningByFriends, users have three main needs that the application must satisfy:
 - want to connect with friends who are also using the application (connect with friends on any social network such as Twitter, LinkedIn, or Facebook)
 - want to create and look at reviews of restaurants and then get recommendations for a restaurant based on these reviews
 - want to rate the reviews of restaurants to indicate whether the review was helpful or not

Example

- Business entity: What are the fundamental building blocks of our application? How are these related to one another?

Example

- Business entity: What are the fundamental building blocks of our application? How are these related to one another?
 - People write reviews.
 - Reviews discuss restaurants.
 - Restaurants serve one/more types of food.
 - A person is friends with another person.
 - People rate reviews.
 - User data - First and last name to help identify users
 - Restaurant data - name, address, types of foods served
 - Reviews -Descriptions of the users' experiences.
 - Ratings – is the review helpful or not?

Example

- Domain and scope: What will the app/the graph do for its users?
- Business entity: What are the fundamental building blocks of our application? How are these related to one another?
- Functionality: How are people going to use the system?

Example

- Domain and scope: What will the app/the graph do for its users?
 - **DiningByFriends** is a social dining application that helps users get personalized restaurant recommendations based on reviews from their friends and other users. The app integrates social networking, restaurant reviews, and recommendation algorithms to enhance the dining experience.
- Business entity: What are the fundamental building blocks of our application? How are these related to one another?
 - People write reviews.
 - Reviews discuss restaurants.
 - Restaurants serve one or more types of food.
 - A person is friends with another person.
 - People rate reviews
 - User data - First and last name to help identify users
 - Restaurant data - name, address, types of foods served
 - Reviews -Descriptions of the users' experiences.
 - Ratings – is the review helpful or not?
- Functionality: How are people going to use the system?
 - Who are my friends?
 - Who are the friends of my friends?
 - How is user X associated with user Y ?
 - What restaurant near me with a specific cuisine is the highest rated?
 - Which restaurants are the 10 highest-rated restaurants near me?
 - What are the newest reviews for this restaurant?
 - What restaurants do my friends recommend?
 - Based on my friends' review ratings, what are the best restaurants for me?
 - What restaurants have my friends reviewed or rated in the past X days?

2 - Create the conceptual model

- Identifying and grouping entities
- Identifying relationships between entities

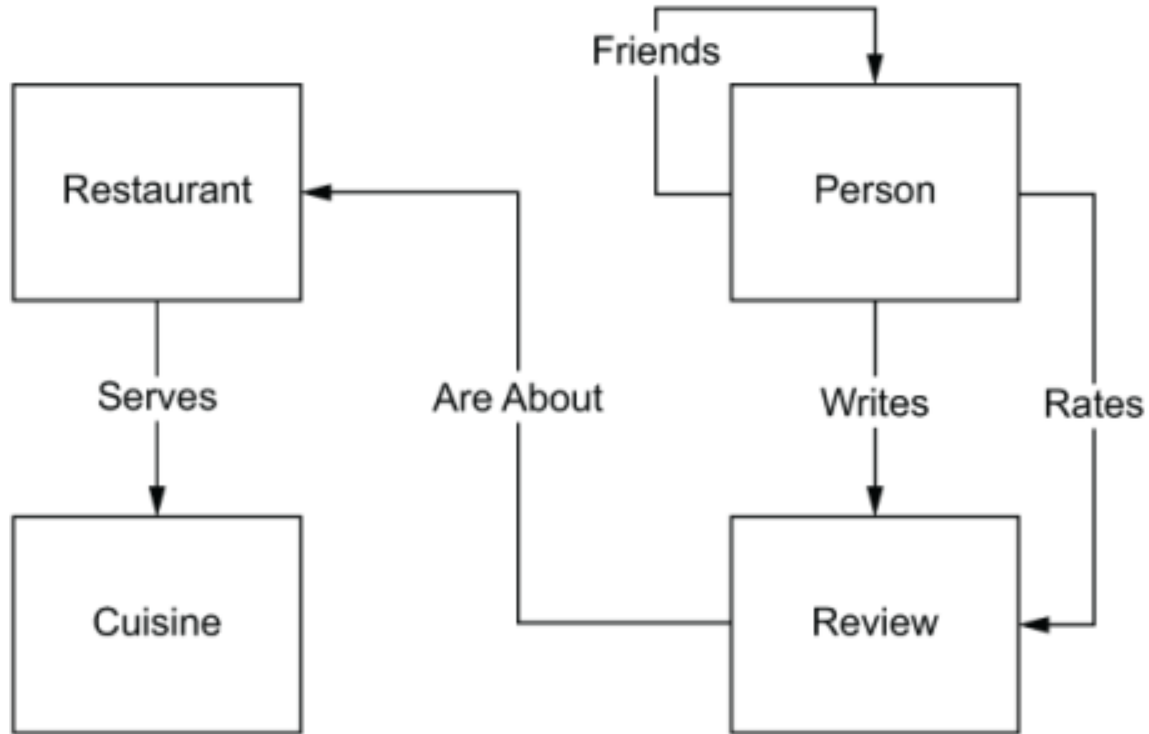
Example

- Identifying and grouping entities
- Identifying relationships between entities

Example

- Identifying and grouping entities
 - Restaurant --Represents a restaurant, which includes name and location.
 - Cuisine --Describes the type of food served. This entity was not explicitly defined as one of the nouns but was found by listening to how the business described its needs.
 - Person --Represents a system user, which includes the first and last name.
 - Review --Actual review content, which includes the full review text and rating.
- Identifying relationships between entities
 - Person-Friends-Person-This relationship helps construct the social network component of our app by allowing the application to answer user questions such as
 - Who are my friends?
 - Who are the friends of my friends?
 - How is user X associated with user Y ?
 - Person-Writes-Review -This relationship enables us to construct the recommendation engine for DiningByFriends as the reviews serve as the underlying data to provide recommendations. This relationship allows the app to answer user questions such as
 - Which restaurant near me with a specific cuisine is the highest rated?
 - Which restaurants are the ten highest-rated restaurants near me?
 - What are the newest reviews for this restaurant?
 - What restaurants do my friends recommend?

Example



3 - Building a Graph data model

There are four steps to building a (logical) graph data model from a conceptual model:

- 3.1 Translate entities to vertices.
- 3.2 Translate relationships to edges.
- 3.3 Find and assign properties to vertices and edges.
- 3.4 Check your model.

3.1 Translating entities to vertices

The entities in a conceptual data model map almost directly to the vertices in a logical graph model. The creation of the vertices in our graph model requires two things:

- Identify all the relevant entities from our conceptual model
- Give the vertex a name in the form of a label to uniquely identify that type of entity in our graph model

It is a best practice to make vertex labels singular because each vertex only refers to a single instance of an item.

3.2 Translating relationships to edges

Edges are based on the relationships from our conceptual model. Defining edges takes a little more effort than finding the vertices.

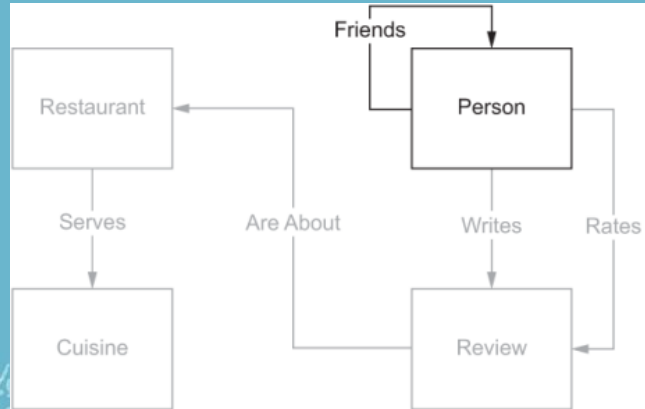
The edges in graph databases include features like directionality and uniqueness, which do not have direct counterparts in relational databases.

- **Identifying** the relevant relationships from the conceptual data model
- **Naming** the edge in the form of a label to uniquely identify that relationship in our graph data model
- Assigning a **direction** to the edge by defining the start and end vertex types
- Specifying the **uniqueness** of the edge by deciding on the number of times this edge can exist between two specific instance vertices

3.2.a – Identifying the relevant relationships from the conceptual data model

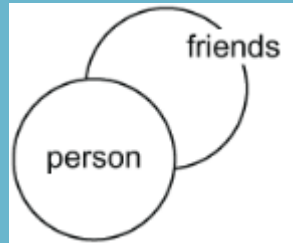
Functionality questions for the social networking we discovered while building our conceptual model:

- Who are my friends?
 - Who are the friends of my friends?
 - How is user X associated with user Y ?
- All the questions revolve around how one person is connected to another as a friend.
 - The Friends relationship is the link from a person to a person vertex



3.2b Naming the edge labels

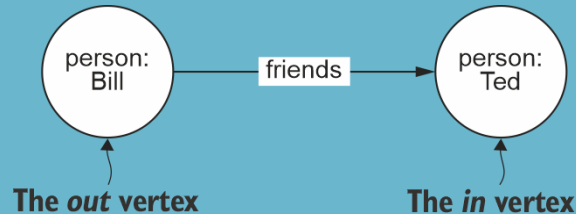
- Now that we know that we need to represent the Friends relationship as an edge in our model, it is time to name it, just like we did with vertices.
- To decide on the edge label for our data model, we apply the same best practice naming rules: be **concise, descriptive, and generic**. By applying these rules, we get an edge with a friends label that starts at a person vertex and ends at a person vertex



3.2.c Giving the edge a direction

Once we have a label for our edge, the next step is to give the edge a direction.

By convention, the direction of an edge is described as being from one vertex, *the out vertex*, to another vertex, *the in vertex*.

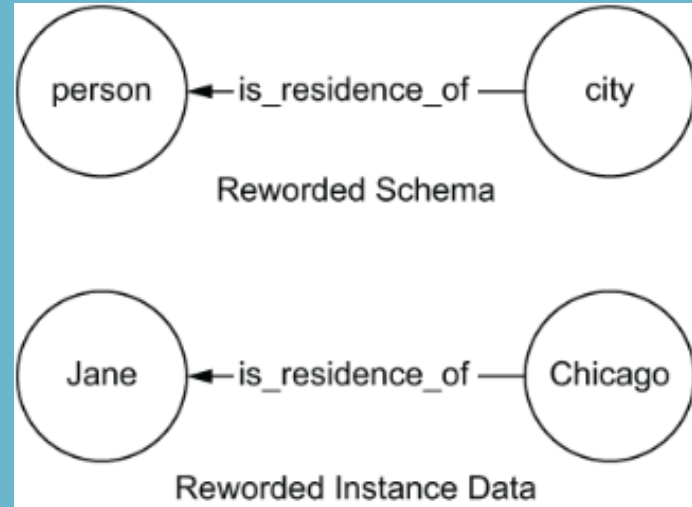
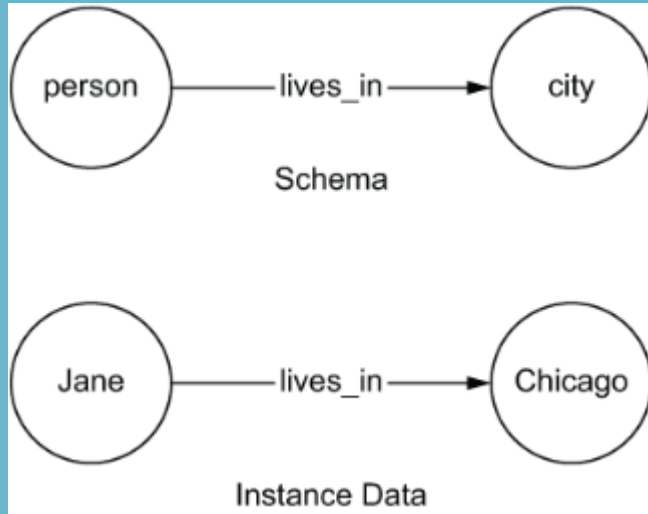


In a good graph model, the vertex-edge-vertex combinations read as a sentence.

Bill friends Ted

Example

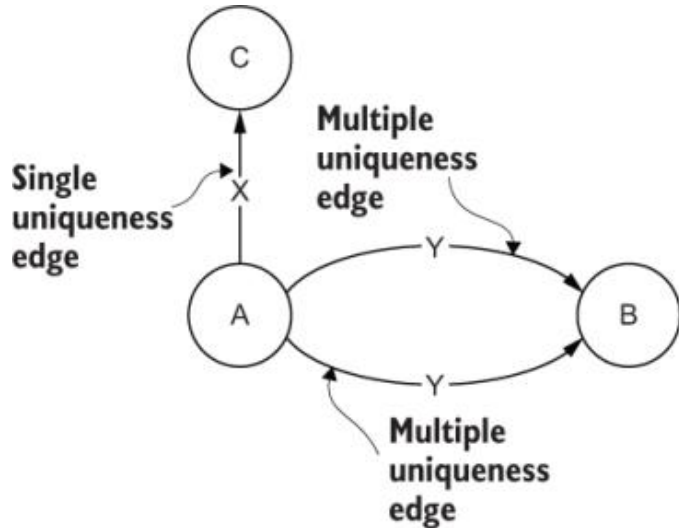
You should reword the label or switch the direction of the edge to make the data model more understandable. The direction of an edge should complement the edge label to make a sentence that sounds natural (or mostly natural) and that fits the functional needs of the use case.



3.2.d Determining edge uniqueness

Edge **uniqueness** describes the number of times an instance of a vertex is related to another instance of a vertex with an edge having the same label.

Uniqueness describes what is the allowable number of edges of a given label between two vertices.



WHY UNIQUENESS AND NOT CARDINALITY OR MULTIPLICITY?

The term cardinality is often used (wrongly) to refer to a many-to-many or a one-to-many relationship.

NEW DEFINITIONS:

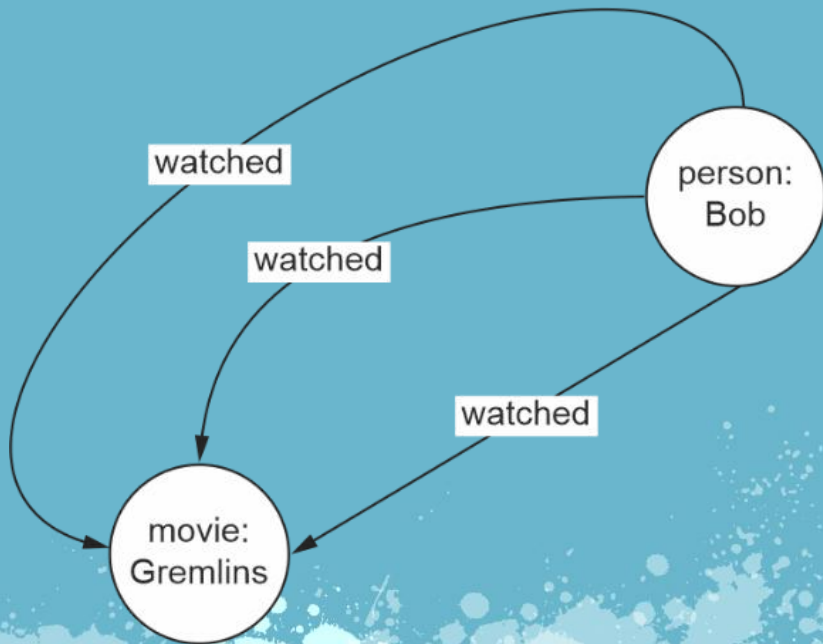
Cardinality is the number of elements in a set (for example, has two Y edges between A and B).

Multiplicity is a specification of the minimum and maximum cardinality that a set can have (for example, one-to-many, zero-to-many, many-to-many).

Data uniqueness describes the measure of duplication of identified data items within a data set. We define uniqueness as the allowable number of edges of a given label between two vertices.

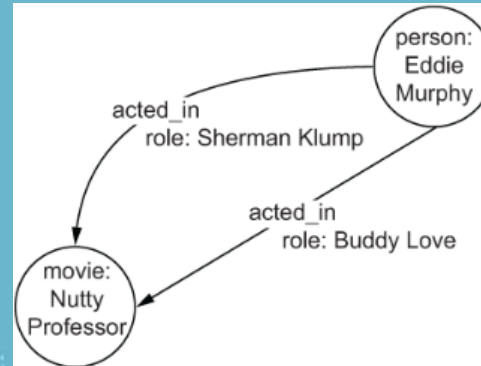
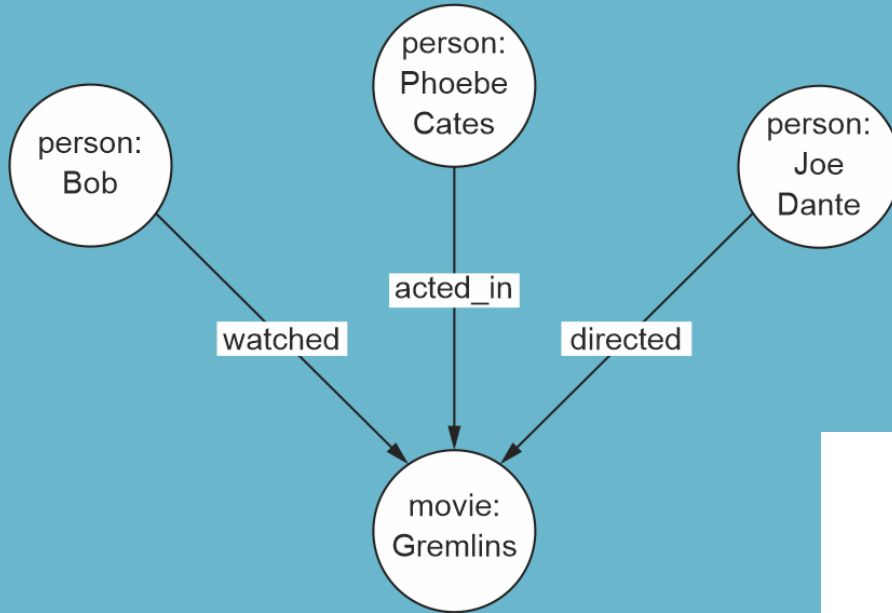
Example

- Uniqueness is the allowable number of edges of a given label between two vertices.
- So single uniqueness refers to zero or one edge, and multiple uniqueness means more than one possible edge (similar to a unique constraint on a SQL column).



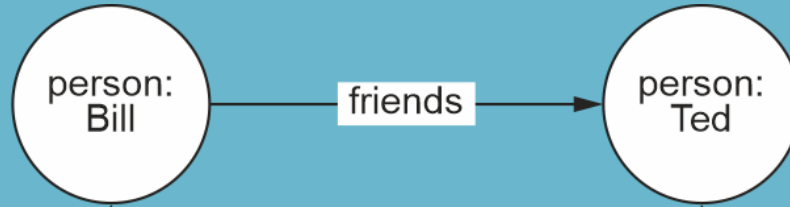
Multiple uniqueness demonstrated with multiple ***watched*** edges between Bob and Gremlins

Example



Adding a role property to the ***acted_in*** edge creates a multiple uniqueness edge to express the fact that *Eddie Murphy* acted in multiple roles in the movie *Nutty Professor*.

Example

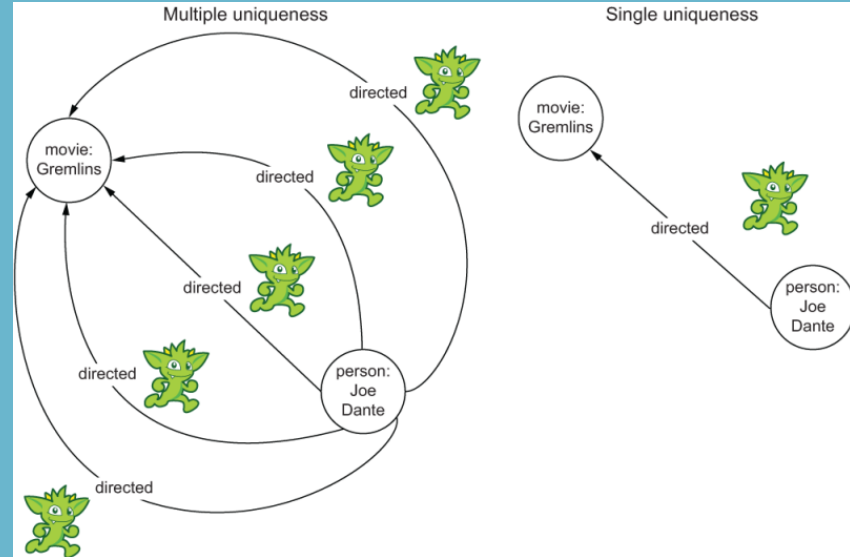


SINGLE UNIQUENESS O MULTIPLE UNIQUENESS EDGE?

Improper uniqueness

Incorrect uniqueness can affect an application:

1. Too little data returned: when we have an edge with single uniqueness that really should be multiple.
2. Duplicated data returned: when we have multiple uniqueness edges but we should only have one.
3. Poor query performance: [harder to diagnose]
Most often having a multiple uniqueness edge instead of a single uniqueness edge is what causes poor query performance, because the database has to do more work to return the data for a query with multiple edges.



3.3 Find and assign properties to vertices and edges

Properties in a graph data model are key-value pairs that describe a specific attribute of a vertex or edge.

Before we can assign the properties to edges and vertices:

- What properties are required?
- How we are going to name them?
- What is their data type?

To answer these questions, we need to consider what we know about the domain as well as our conceptual data model to decide what information needs to be stored.

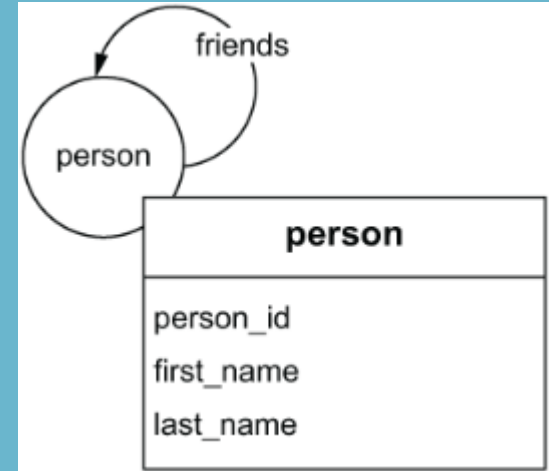
DEFAULT AND NULL VALUES IN GRAPH DATABASES?

Properties in a graph DB are similar to the columns in a row in a relational DB table: these stores the data about a specific entity.

Unlike columns, an application does not insert default values nor null values into properties in graph databases.

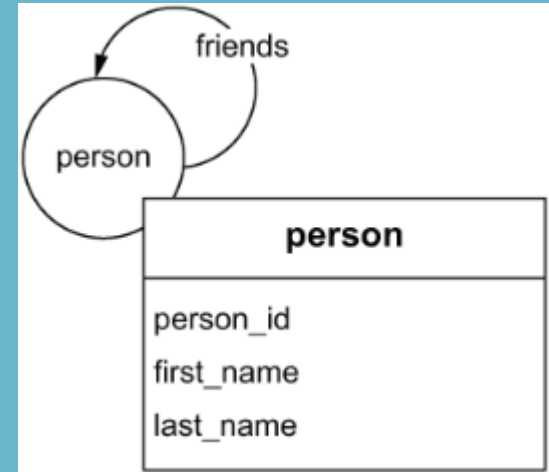
Example

- Looking at the functionality questions, these are
 - Who are my friends?
 - Who are the friends of my friends?
 - How is user X associated with user Y ?
- Based on these questions, we can see that we need to store the first_name and last_name for a person to identify who our friend is. We can also assume that we are going to need a unique identifier of each person (person_id) to differentiate between people with the same name. Without this additional attribute, we are unable to discern one John Smith from another John Smith.



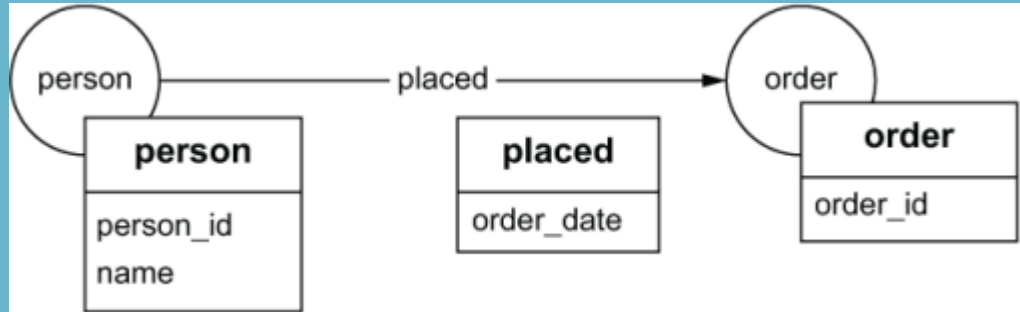
Example

- Looking at the functionality questions, these are
 - Who are my friends?
 - Who are the friends of my friends?
 - How is user X associated with user Y ?
- Based on these questions, **What are the properties for a person to identify who our friend is?**



Example

- Similarly we can set properties on edges
- **Weights** are examples of edges' properties



At this point in the process, if you chose to use any of the schemaless graph databases that exist on the market, then you are complete.

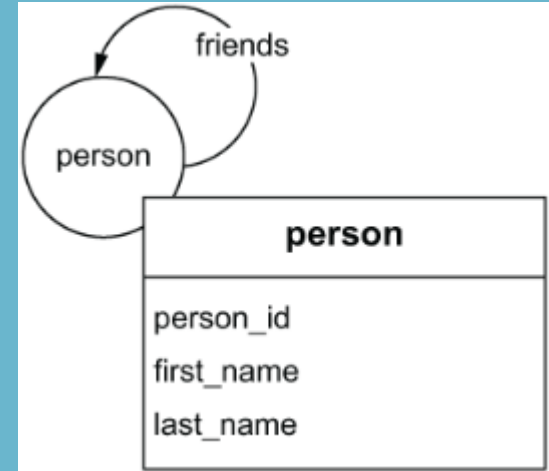
But if you are using a database that requires the schema to be explicitly specified, then you have one additional step: translating your logical data model into the **physical data model** required by your chosen database.

3.4 Check your model

The last step in creating our logical data model is to **validate** that we can answer the questions for our social network use case and that the model we built follows best practices for graph data modeling.

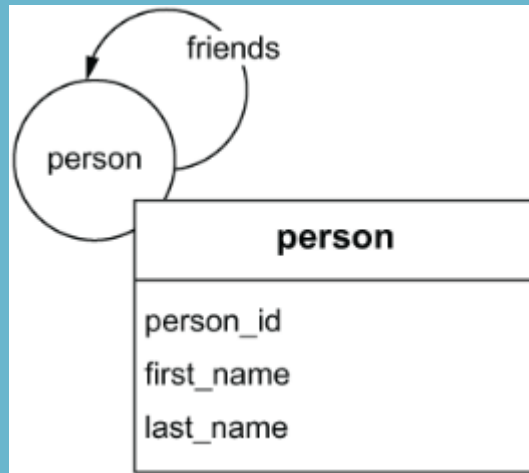
Example

- Looking at our questions, can we answer these using our graph data model?
 - Who are my friends?
 - Who are the friends of my friends?
 - How is user X associated with user Y?



Example

- Looking at our questions, can we answer these using our graph data model?
 - Who are my friends? We can answer this question by starting at a specific person found by `person_id` and traversing the friends edge to see all of their friends.
 - Who are the friends of my friends? We can answer this question by starting at a specific person found by `person_id` and traversing the friends edge, and then traversing the friends edge again to see all of the friends of my friends.
 - How is user X associated with user Y? We can answer this question by starting at a specific person found by `person_id` and traversing the friends edge until we either have no more friends edges to traverse or we traverse to the destination person.



Best practice checks

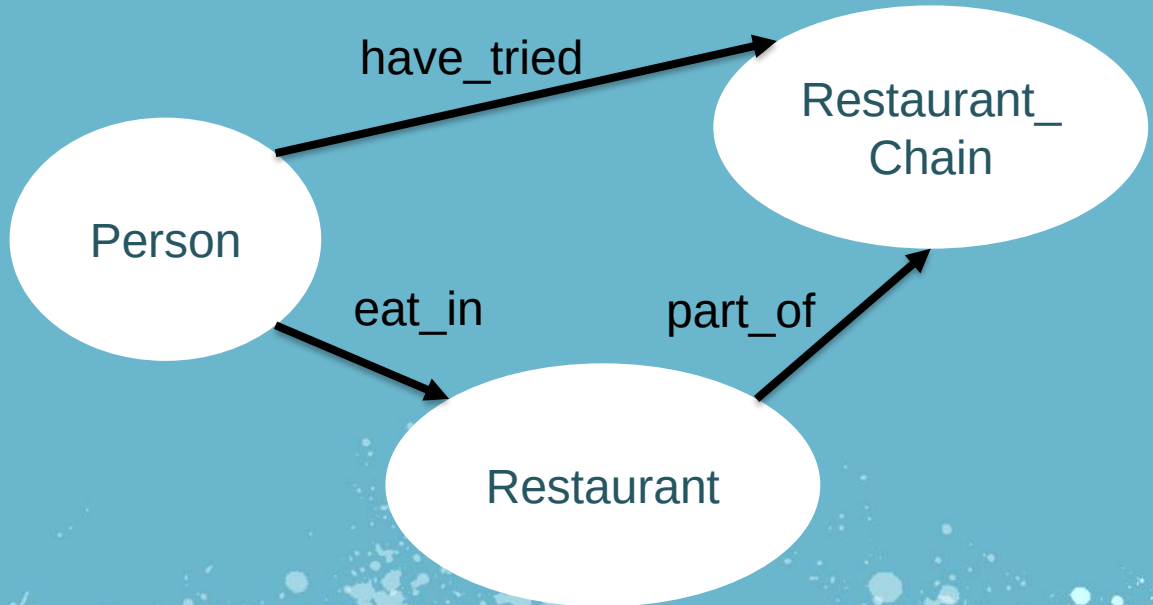
- Few additional best practice checks to make sure that our data model provides a solid graph model:
- Do the vertices and edges read like a sentence? Yes. While this is not an absolute requirement, it is an excellent general check to verify that vertex labels represent the nouns in your model and the edge labels describe the actions or verbs in your model.
- Do I have different vertex or edge labels with the same properties? No. In this case, we only want a single vertex and a single edge label.
- Does my model make sense? Yes. While this step can seem like an obvious check, it does pay to take time to step back and double-check that your graph data model has not strayed too far from your conceptual data model and that it makes sense for the problem that you are solving.

4 - Test your model

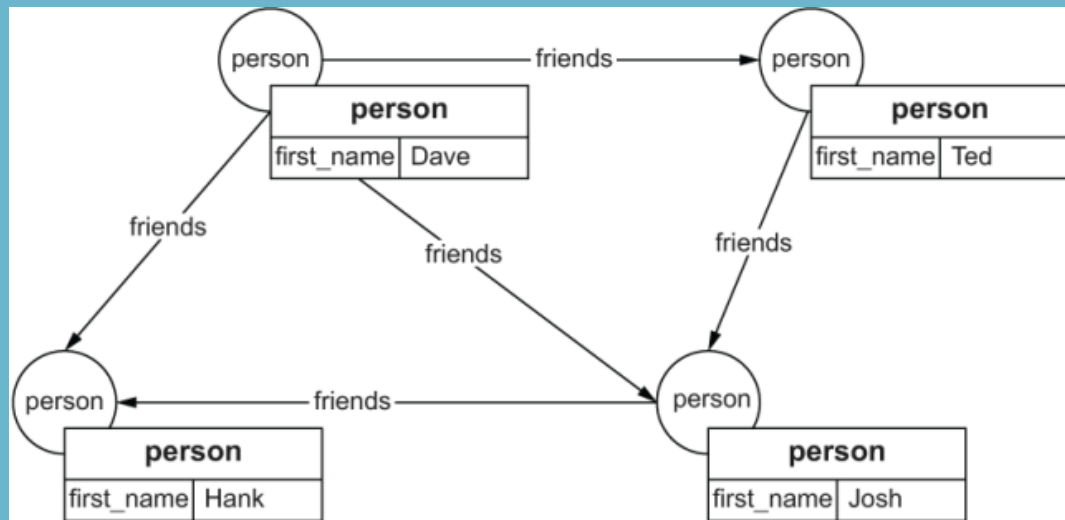
Now start using the model and see if modifications are needed

EXAMPLE

Adding direct relationships if needed



The social network graph (in Neo4j)



```
CREATE (a:Person {first_name: 'Dave'}), (b:Person {first_name: 'Ted'})
```

```
CREATE (c:Person {first_name: 'Josh'})
```

```
CREATE (d:Person {first_name: 'Hank'})
```

```
MATCH (a:Person), (b:Person)
WHERE a.first_name = 'Dave' AND b.first_name = 'Ted'
CREATE (a)-[r:FRIENDS]->(b)
RETURN type(r)
```

...

TODO @HOME

- Create a complete graph for modeling *DiningByFriends* in Neo4j
- Add instances at will
- Show the schema to check if it match the conceptual model originally defined
- Navigate the graph
- Answer **Functionality questions**

Additional readings

- Graph data Modelling
<https://opencredo.com/blogs/ingesting-big-data-into-neo4j-part-1>



Thank you

Insert the title of your subtitle Here